

ResNet

Deep Residual Learning for Image Recognition – Kaiming He, Xiangyu Zhang

- HEART LAB 세미나 3주차 ResNet 논문 리뷰

HEART LAB 학석연구생
인공지능학과 유건

목차

1. Introduction : Why ResNet?

- Why Deep Neural Network?
- Previous Model
- Degradation of training accuracy

2. Deep Redidual Learning

- Residual Structure
- Residual Block
- Network Architectures

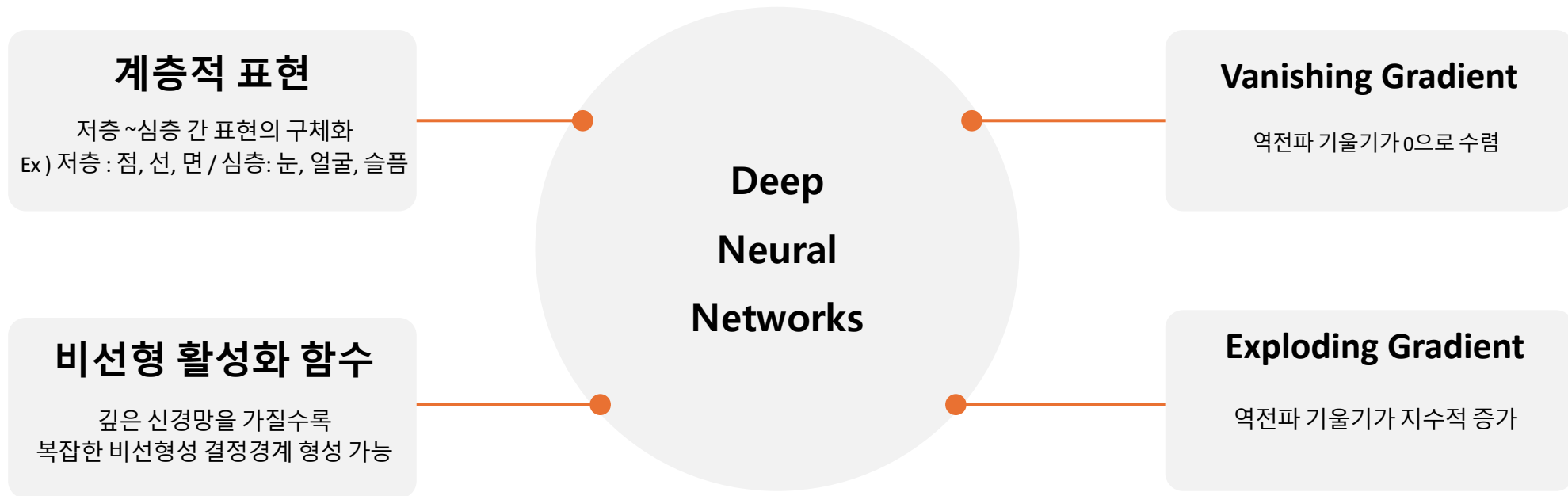
3. Experiments

- Imagenet Classification
- Deeper Bottleneck Architectures

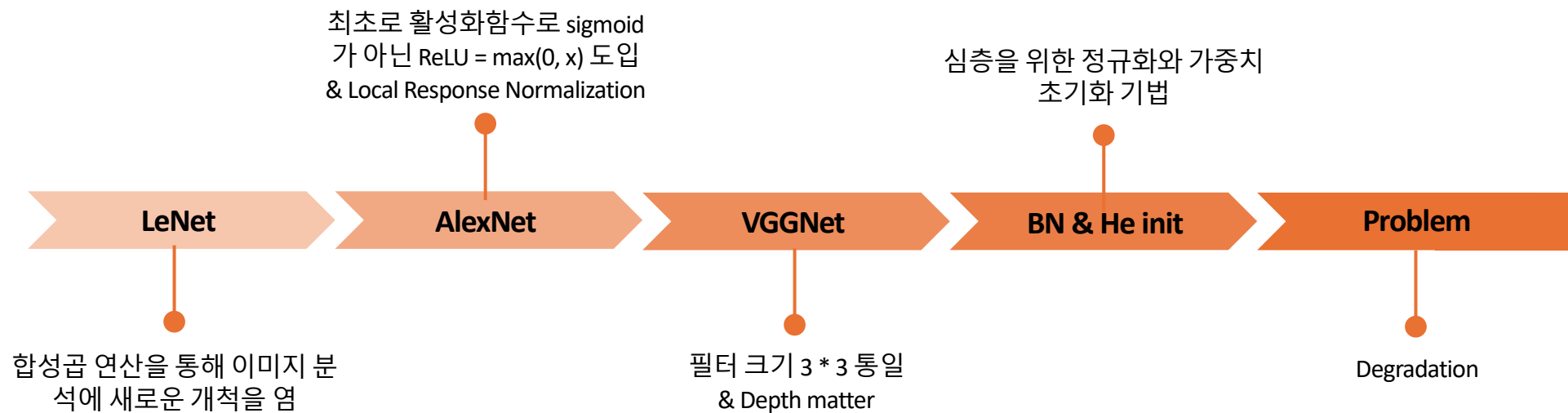
4. Conclusion

5. Q & A

Why ResNet?



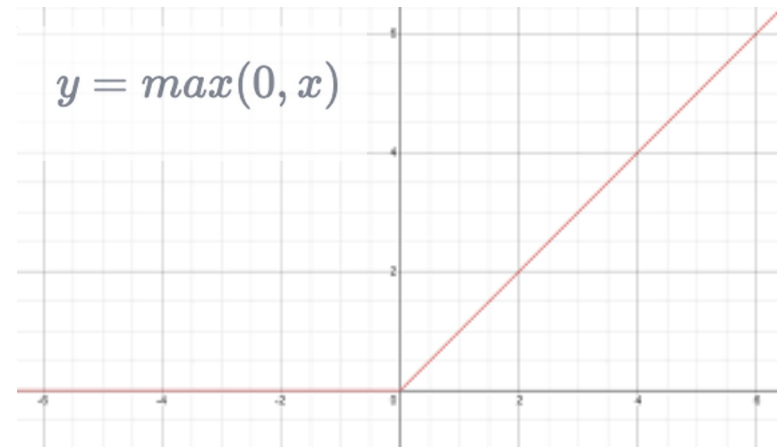
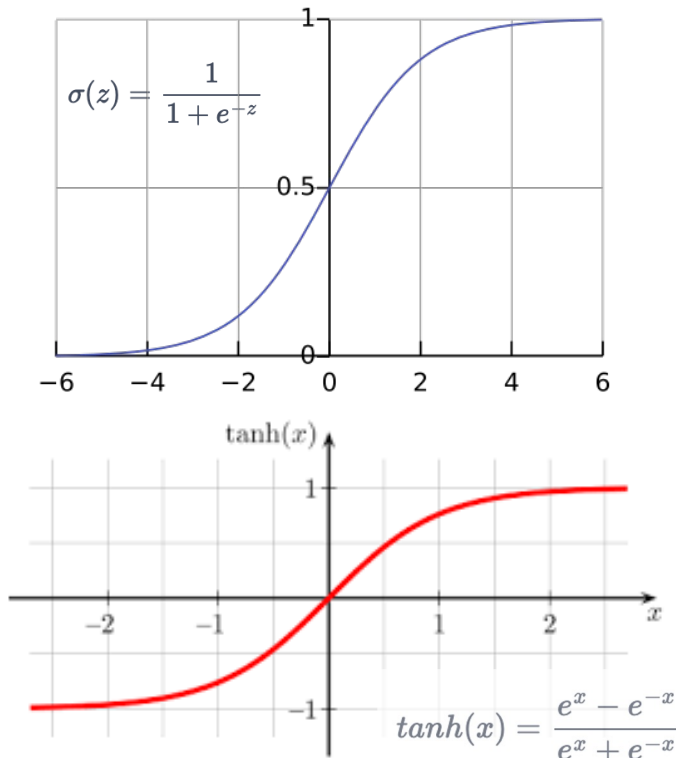
Why ResNet?



Why ResNet?

AlexNet : ReLU & Local Response Normalization

1. ReLU (Active function)



- 치역이 $(0,1)/(-1,1)$ 에서 $(0,\infty)$ 로 바뀌어서 Vanishing Gradient 완화!

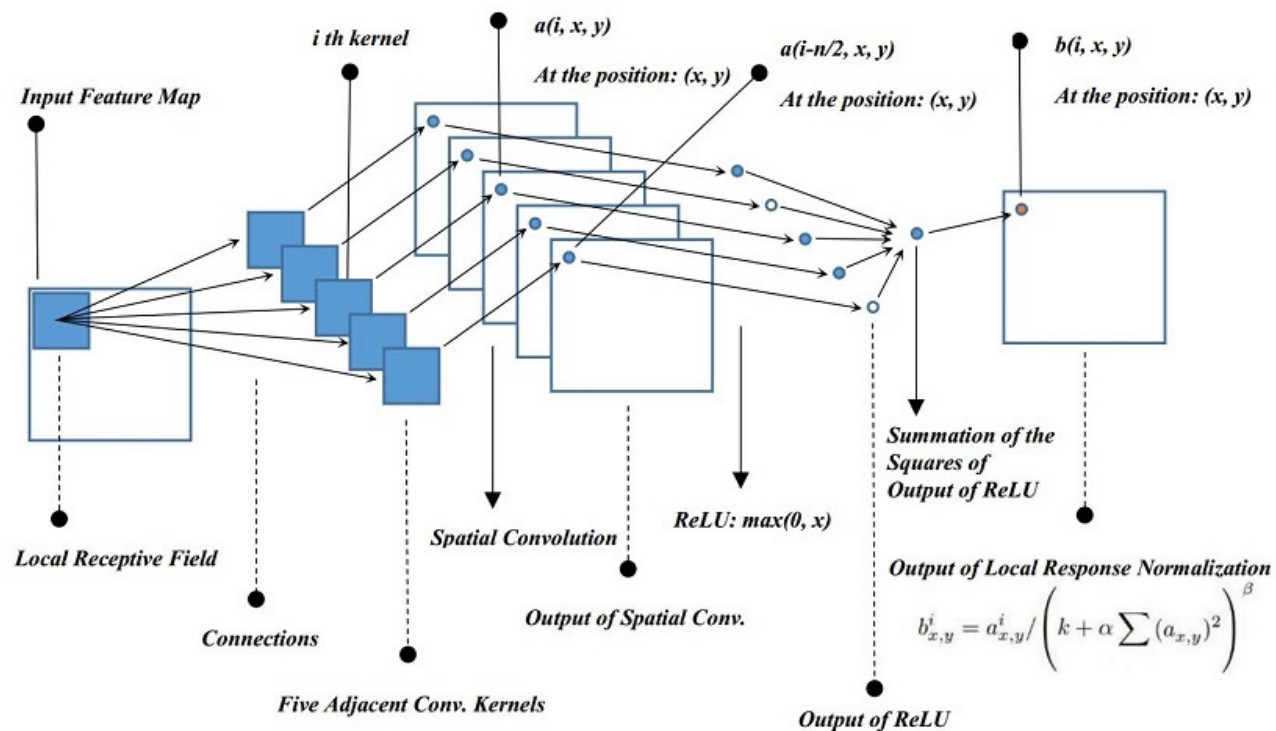
Why ResNet?

AlexNet : ReLU & Local Response Normalization

2. Local Response Normalization

$$b_{i,x,y} = \frac{a_{i,x,y}}{(k + \alpha \sum_{j=\max(0, i-\frac{n}{2})}^{\min(N-1, i+\frac{n}{2})} (a_{j,x,y})^2)^\beta}$$

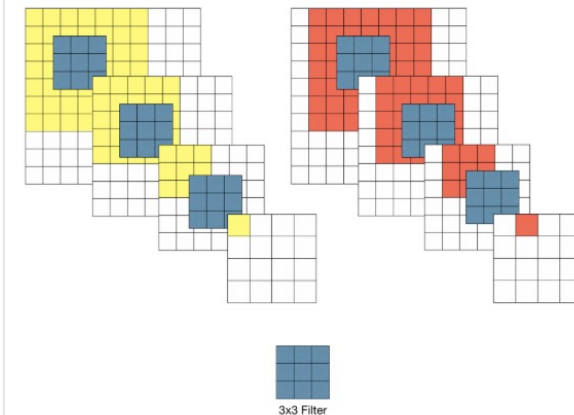
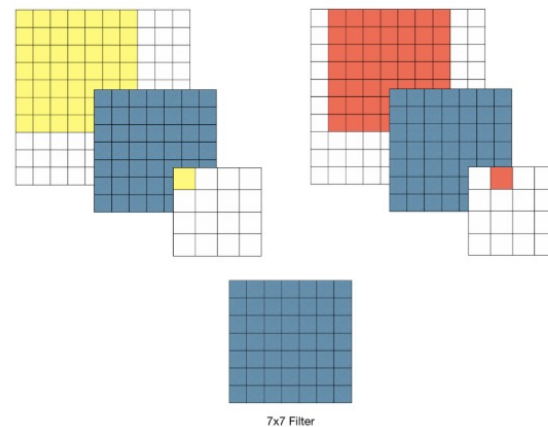
- 국소적 정규화를 통해 값이 큰 특정 채널의 강한 활성화!



Why ResNet?

VGGNet : Simple Architecture & Depth matter

- 7 * 7 필터 Conv보다 3 * 3 필터 3 번 Conv 시 많은 Layer로 학습 가능하며 파라미터 수 또한 더 적다. ($7 * 7 : 49$, $3 * 3 * 3 = 27$)
- 작은 필터를 사용함으로써 더 많은 ReLU함 수를 사용할 수 있고 더 많은 비선형성의 확보 가능



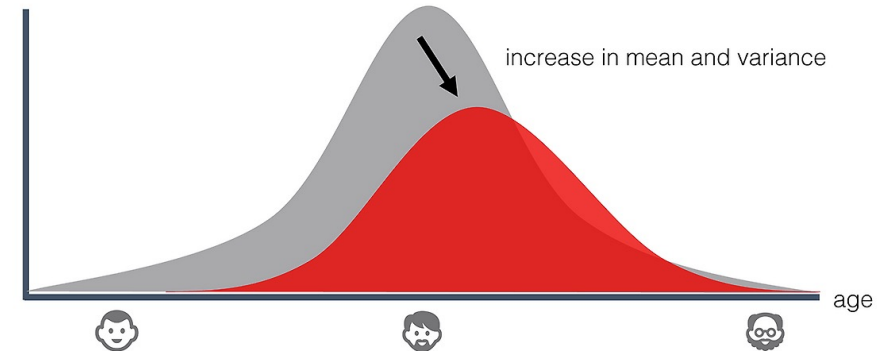
Why ResNet?

Batch Normalization & He initialization

1. Batch Normalization

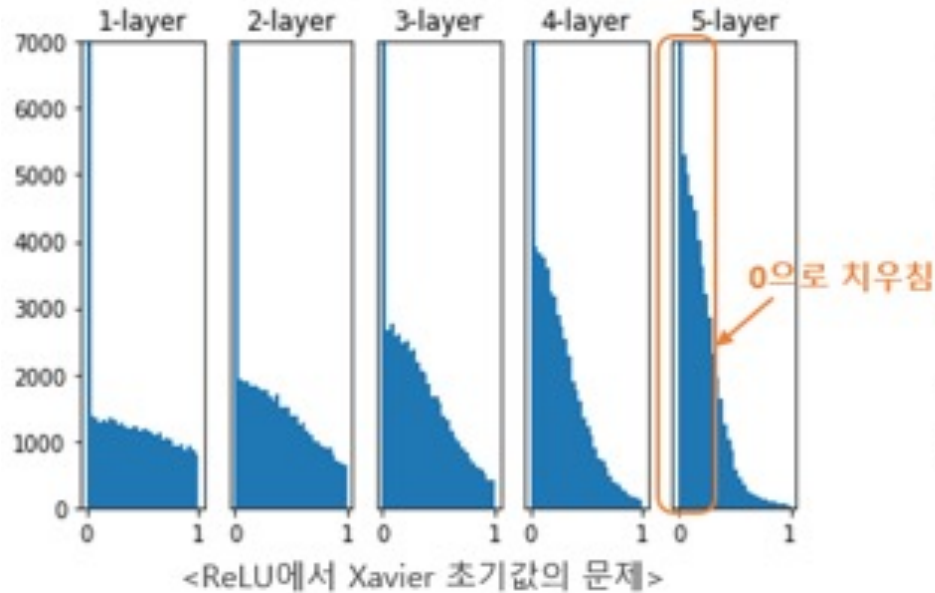
$$\hat{x}^k = \frac{x^k - \mu_B^k}{\sqrt{(\sigma_B^k)^2 + \epsilon}} \quad \& \quad y^k = \gamma^k \hat{x}^k + \beta^k$$

- 입력 분포를 일정하게 만들어주어서
심층에서 Exploding Gradient 완화!



Why ResNet?

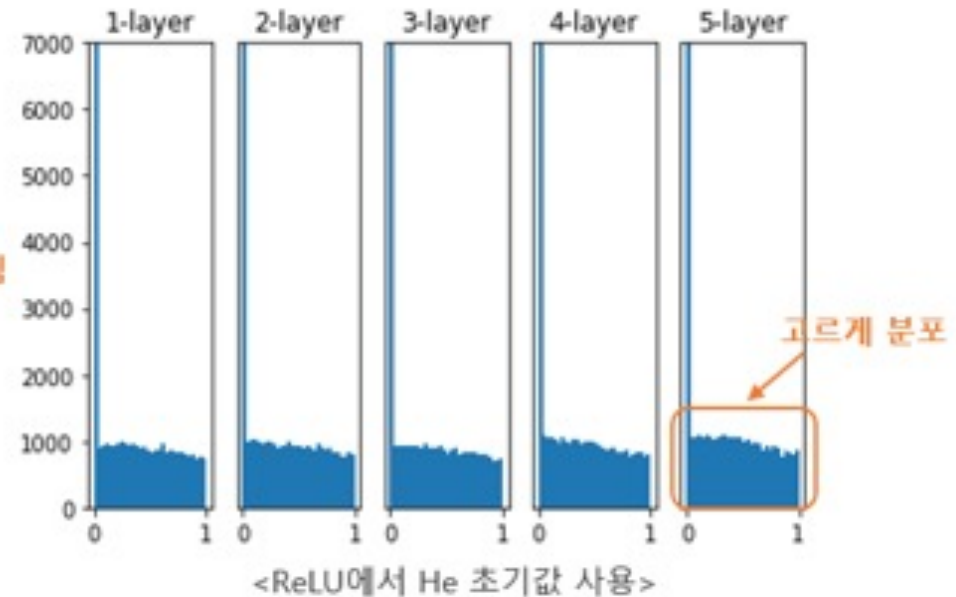
Batch Normalization & He initialization



Xavier init

$$W_{ij} \sim \mathcal{N}(0, \frac{1}{n_{in} + n_{out}})$$

2. He initialization



He init

$$W_{ij} \sim \mathcal{N}(0, \frac{2}{n_{in}}) \quad | \quad n_{in} = \text{입력노드수}, n_{out} = \text{출력노드수}$$

Why ResNet?

Problem : Degradation

- 신경망은 보통 층을 더 깊게 쌓으면 더 복잡한 함수를 학습할 수 있어서 성능이 좋아질 것으로 기대
- 실험 결과, 어떤 시점부터는 층을 더 추가했는데도 오히려 정확도(training accuracy/test accuracy)가 감소
- 이는 과적합(over fitting) 때문이 아닌 심층으로 갈수록 Optimizing의 어려움 때문

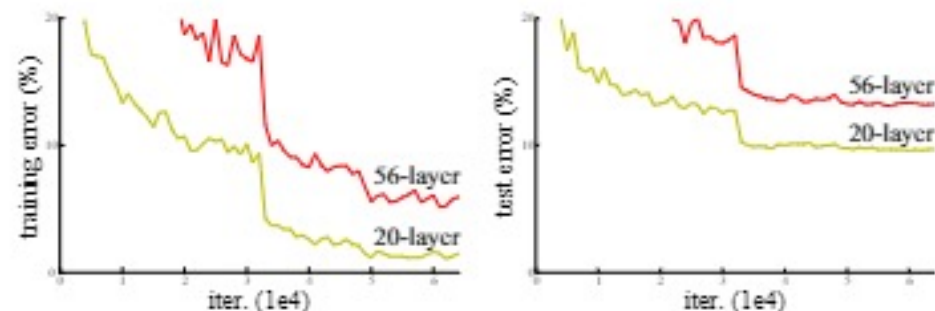


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

What is ResNet?

“There exists a solution by construction to the deeper model: the added layers are identity mapping, and the other layers are copied from the learned shallower model.

The existence of this constructed solution indicates that a deeper model should produce no higher training error than its shallower counterpart”

-As He et al. (2015) noted, Deep Residual Learning for Image Recognition

- 만약 얇은 모델에 Layer를 추가해서 깊은 모델로 만들었을 때 그 Layer가 $y=x$ 라면 Optimizer의 성능이 나빠질 이유가 없다.
- $\text{ReLU}(Wx + b)$ 층으로는 단순히 입력값을 그대로 출력하는 $y = x$ 를 나타내기 어렵고 이는 Optimizer의 성능을 악화시킨다.

Residual Deep Learning

1. Residual Structure

$$y = x + F(x)$$

- 두개의 Layer와 지름길 연결을 해주는 항등 매핑($y = x$) 합해줌

$$F(x) = W_2 * ReLU(W_1 x)$$

- 두 개의 선형결합 계층을 나타냄

$$H(x) = ReLU(F(x, W_i) + W_s x)$$

- 입력과 출력 크기가 같을 땐 x
크기가 다를 땐 $W_s * x$

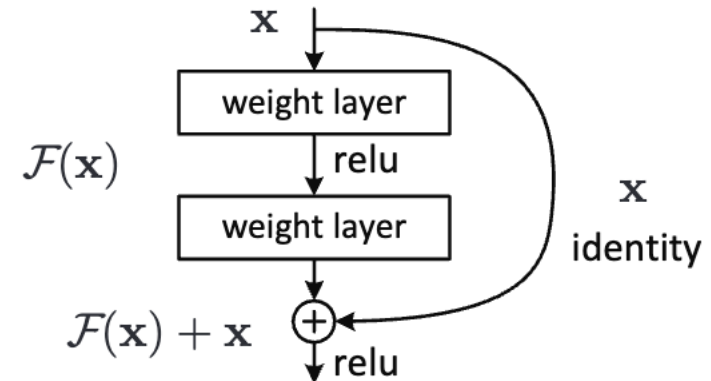


Figure 2. Residual learning: a building block.

Residual Deep Learning

2. Residual Block

$$x \in R^{C*H*W}, y \in R^{C*H*W}$$

$$z_1 = ReLU_1(Conv_1 x)$$

$$z_2 = ReLU(BN_2 Conv_2(z_1))$$

$$y = ReLU(z_2 + x)$$

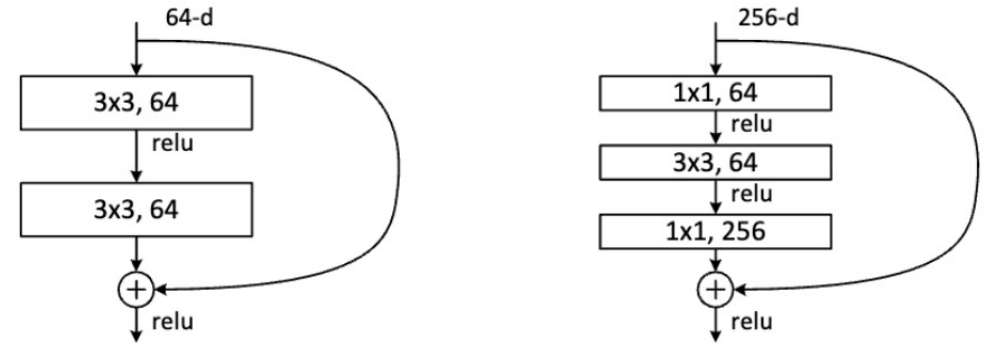


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

Residual Deep Learning

2. Residual Block

In 2016, "Identity Mappings in Deep Residual Networks"

$$y = \text{ReLU}(\mathcal{F}(x) + x) \rightarrow y = x + \mathcal{F}(\text{BN}(\text{ReLU}(x)))$$

Residual Deep Learning

3. Network Architectures

DATASET : Imagenet

1. Image 가로, 세로 비율을 유지한 채 짧은 변의 길이를 (256, 480) 사이의 무작위 크기의 픽셀 사이즈로 변경
2. 크기가 조정된 이미지에서 224 * 224 크기의 패치로 crop 함.
3. 모든 Conv 연산 이후 Batch Normalization 수행함

$$o = \left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1$$

1. Filter = 3 * 3으로 고정
2. Conv 연산 후 BN 진행
3. 입력값과 출력값이 같은 경우 stride = 1, padding = 1
4. Down sizing 진행 시 출력값은 입력값의 절반 크기로 감소하며 채널 수는 두배로 늘어남 따라서 stride = 2, padding = 1

Residual Deep Learning

3. Network Architectures

1. 첫 번째 Layer의 $7 * 7$ Conv

2. Down Sizing 기법 3가지

3. 마지막 계층 Avg Pooling

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	$7 \times 7, 64, \text{stride } 2$				
conv2_x	56×56	$3 \times 3 \text{ max pool, stride } 2$				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Residual Deep Learning

3. Network Architectures - Why 7 * 7 Conv in first Layer of ResNet?

1. 짝수 크기 커널의 구조적 문제점

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \leftrightarrow \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 5 & 6 & 7 \end{bmatrix}$$

- 홀수 커널은 짝수 커널에 비해 명확한 중심점 존재함 = 공간 정보 왜곡 없음

2. Same Padding 시 padding 연산 이점

4 * 4 feature map에 Same Padding으로 3 * 3 filter와 2 * 2 filter Conv 수행

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 & 0 \\ 0 & 5 & 6 & 7 & 8 & 0 \\ 0 & 9 & 10 & 11 & 12 & 0 \\ 0 & 13 & 14 & 15 & 16 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \leftrightarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 \\ 0 & 5 & 6 & 7 & 8 \\ 0 & 9 & 10 & 11 & 12 \\ 0 & 13 & 14 & 15 & 16 \end{bmatrix}$$

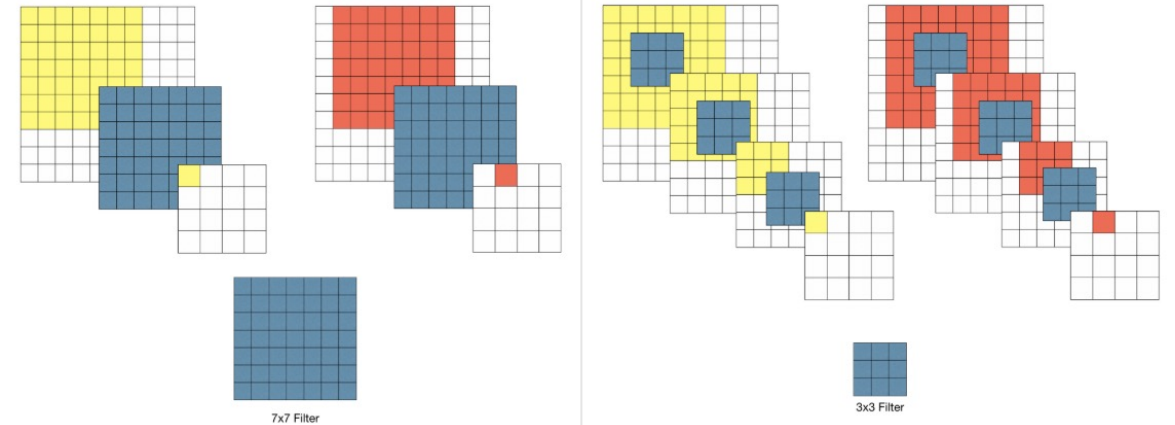
- 홀수 커널과 짝수 커널의 Same Padding 시 Padding 후 Conv 연산 하기 전 데이터 모양

Residual Deep Learning

3. Network Architectures - Why 7 * 7 Conv in first Layer of ResNet?

2. 넓은 수용 영역의 의미

- 7 * 7을 사용하면 3 * 3보다 넓은 영역을 한번에 볼 수 있으며 이는 네트워크의 초기 단계에서 이미지의 형태, 색상분포와 같은 일반적인 맥락을 이해하는 데 결정적인 역할을 함
- 9 * 9, 11 * 11 커널은 파라미터 연산량 대비 성능 증가가 미미하며 5 * 5는 넓은 영역을 보지 못하여 최적의 7 * 7 커널이 채택

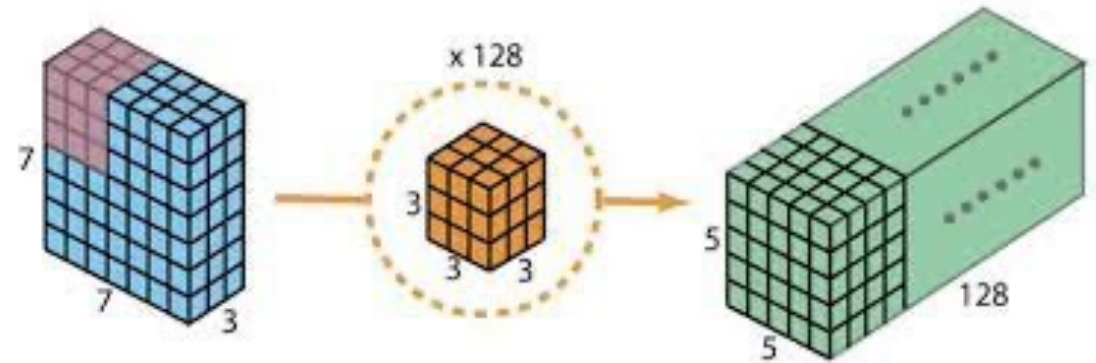


Residual Deep Learning

3. Network Architectures - Why 7 * 7 Conv in first Layer of ResNet?

3. 첫 번째 Conv 연산 시의 이점

- RGB 파일의 입력은 $224 * 224 * 3$ 이므로 다른 Conv 연산된 입력값 $56 * 56 * 64$ 에서 $7 * 7$ 연산을 하는 것 보다 입력 채널, 출력 채널이 작다.
- 처음 : $7 * 7 * 3 * 64$, 나중 : $7 * 7 * 64 * 64$

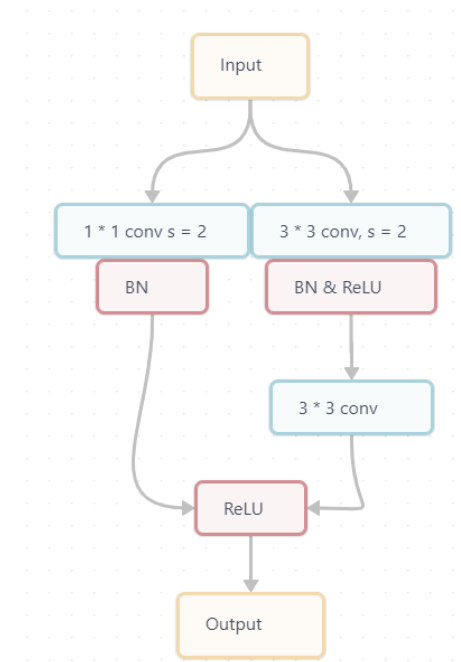


Residual Deep Learning

3. Network Architectures – Down Sizing method

항목	A	B	C
Shortcut 경로 Conv 여부	X	다운 사이징 시 1 * 1 Conv	항상 1 * 1 Conv
Spatial 크기 감소 방법	stride = 2 Avg Pooling 혹은 subsample	1 * 1 Conv with stride = 2	1 * 1 Conv with stride = 2
채널 수 증가 대응			
추가 파라미터	없음	Down sizing projection 에서만 존재함	모든 블록에서 존재함
계산량	매우 작음	적절함	매우 큼
성능	실용적, 손해 있음	정확함, 효율성 균형적	약간 더 나은 성능 계산 효율 떨어짐

B 의 경우 (50 이상의 Layer에서 사용)



Residual Deep Learning

3. Network Architectures – Avg Pooling, 1000-FCL, Softmax

- 마지막 $7 * 7 * 2048$ feature map에 대해 $7 * 7$ Average Pooling 수행하면 각각의 채널마다 전체 영역 평균값 1개만 남음
- Edge나 pattern을 추출하기 위한 Max Pooling보다 일반화 성능을 위해 Avg Pooling을 사용
- 따라서 Flatten 없이도 공간 축소 + 채널 유지를 한번에 수행
- 2048크기의 입력값을 FCL 통과 후 1000크기(클래스)의 출력값으로 나타낸 후 Softmax를 통과시켜서 확률값에 대해 나타냄.

Experiments

1. Imagenet Classification : plane vs ResNet

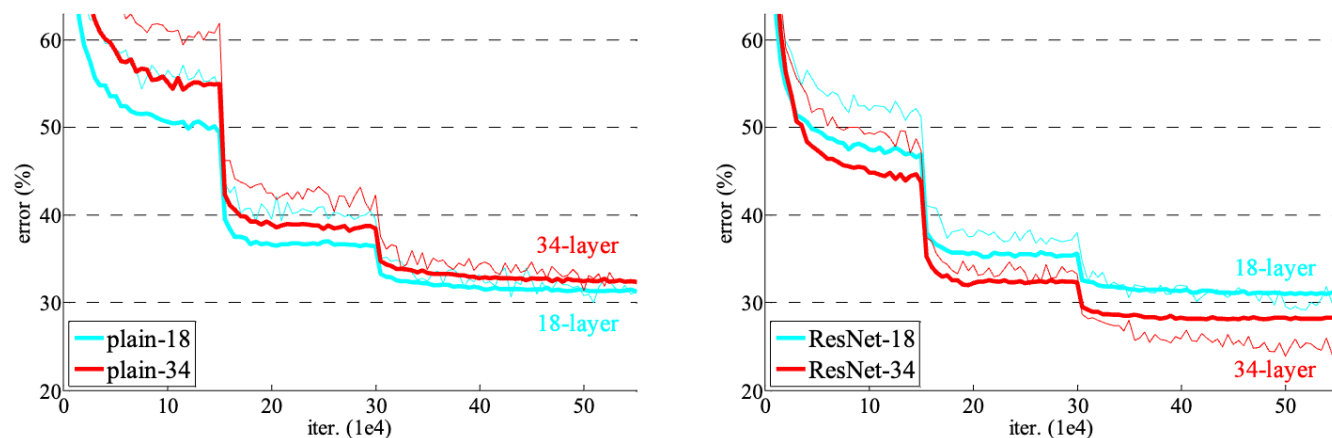


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

Experiments

2. Imagenet Classification : Deeper Bottleneck Architectures

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PRReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

Conclusion

1. Deep Neural Network에서 Vanishing/Exploding Gradient를 해결했으나 Degradation 문제 발생
2. 추가된 Layer가 항등매핑($y = x$)라면 Accuracy가 낮아질 이유가 없으며 이는 Optimizing 과정에서 문제가 발생을 시사함
3. 위의 가설을 근거로 항등매핑 ($y = x$)를 보장하는 잔차학습을 모델 Layer로 사용함으로써 Optimizing 문제 해결
4. 단순히 이미지 분류 모델만이 아닌 잔차 학습의 원리가 다른 심층 모델에서도 일반적인 학습 구조로 적용 가능함을 시사함

감사합니다

- HEART LAB 세미나 3주차 ResNet 논문 리뷰

HEART LAB 학부 인턴과정
인공지능학과 유건